Arvindeepeet Singh - QF2023U1233

Gen - AI

Hugging Face:

Hugging face is an open-source AI platform that provides a large collection of pre-trained machine learning models and datasets that can be used for tasks like text generation, translation, image recognition, and speech processing. It act as a central hub where developers and researchers share AI models similar to how GitHub is used for code. Hugging face makes state-of-the-art AI easily accessible without requiring users to train complex models from scratch.

Hugging face works by allowing users to download pre-trained models and directly apply them to using the transformers library, which automatically handles tokenization, architecture loading, and configuration.

The platform also hosts tools for deploying AI applications and API's for real-time inference. It is used because it save huge computational costs, supports rapid experimentation, and enables and encourages for open collaboration.
Additionally, Hugging face supports fine-tuning, which means users can customise existing model for their own dataset to improve accuracy.

OLLAMA

Ollama is a tool that allow users to run large language models (like LLaMA, Mistral, or Gemma) directly on their local computer instead of using cloud platform/server. It is designed to be light weight and optimized so that powerful AI models can function smoothly even on personal devices. This makes it useful for developers who need privacy, offline access, and full control over their data while experimenting with AI models.

managing model

Ollama works by loading, memory optimization and GPU/CPU utilization automatically in the background. Users simply type a command like "ollama run llama2" and Ollama downloads, configures, and executes the model locally.

It is used because it is simple to use, install, doesn't require cloud accounts, and ensures that all data stays on the user's machine, reduce privacy risks and usage costs. Additionally, it supports custom prompts and model fine-tuning locally to personalise AI responses.

It support multiple models like Mistral, llama, Gemma, etc and use computer resources like CPU, GPU to run on locally.

Pre-training

Pre-training is the first major phase of building a large AI model where the model is trained from scratch on massive amount of general data [like books, website, research paper]. The goal is not to solve a specific task, but to help the model to learn language, grammer, reasoning patterns, and knowledge.

Pre-training is like teaching a child a whole language before asking them to write essay or answer questions. The model reads huge amounts of text and learn patterns, such as:

- → How sentences are formed
- → what words commonly follow others.
- → Concepts about the world.

It learns by predicting the next word in a sentence or filling in missing words. over time, it builds a deep understanging of language.

How pre-training works:

- → Data collection - Trillions of tokens of text are collected.
- → Tokenization → Text is broken into tokens
- → Self-supervised learning - The model learns without labeled ans.
- → Optimization - Using back propagation + gradient decent to adjust billions of parameters.
- → Knowledge formation : the model stores patterns, facts, and reasoning ability in its parameters.

Fine - Tuning

Fine tuning is the second stage after pre-training where a general AI model is specialiseed for a specific task or domain using a smaller, focused dataset like Medical conversation, legal documents, coding, etc.

Fine tuning takes a pretrained model (which is already able to understand language and has general knowledge) and further trains it on specific task's data so it becomes expert in that field.
eg: A general language model can be fine-tuned to become a medical assistant or a coding chatbot.

Working:
1) Start with a pretraining Model
2) Provide Domain specific or Task specific data.
3) Supervised learning → Model learns from pairs of input and correct output.
4) Upgrade parameters slightly.
   → Only a small part of the models weight are adjusted (specially when using LoRA or PEFT)

Types of fine tuning:
→ Full fine-tuning : update all models parameters.

→ PEFT / LoRA Fine-tuning : update only a small no. of parameters.

PEFT and Its types.

PEFT stands for Parameter-Efficient Fine-Tuning. It is a modern technique used to fine-tune large AI models without updating all of their billions of parameter.
Instead of retraining the entire model, PEFT modifies or adds only a small portion of parameters, making fine-tuning: Faster, Cheaper, and possible even on small GPUs

Traditional fine-tuning requires huge memory and compute. PEFT solves this by freezing the base model and learning only small additional weights that adapt the model to a new task.

Types of PEFT Methods:

1) LoRA (low-Rank adaptation):
→ add small trainable parameters inside layers.
→ Only these small matrices are trained, original model is ~~it~~ frozen.
→ very efficient and widely used for LLMs.

2) Q-LoRA (Quantized LoRA):
→ A combination of quantization + LoRA.
→ The model is compressed to lower precision (like-4bit) to fit small GPU's.
→ LoRA adapters are added for fine-tuning.

3) Refix Tuning:-
→ Adds a small set of learned tokens (prefixes) to the input of each transformer layer.
→ The rest of the model remains unchanged.

4) Prompt Tuning:
→ Instead of modifying weights, it learns special prompt embedding.
→ These prompts are prepended to the input has strong general knowledge and to the steer the models behavior.

5) Adapter Tuning:
→ Adds a small neural layer of the model
→ Only these adapters layers are trained.